Chapter 3

자동적으로 움직이는 게임 에이전트 생성법

프로젝트 실행과 기본 설정

- 소스 코드 빌드
  - 프로젝트 속성 창/ vc+++ 디렉토리/포함 디렉토리 에
    - Common 폴더 추가
  - Script1.rc 파일의
    - #include “afxres.h” ->#include “windows.h” 로 변경

프로젝트 실행과 기본 설정

소스는 VS2010의 Chapter3-Steering Behaviors 폴더에 있음

- 포함 경로 Common, 설치된 도구 집합, params.ini 작업 경로 확인
- 기본 실행은 300개 무리와 SHOAL 포식자 설정
- 개별 행동 실험은 다른 플래그와 SHOAL을 먼저 정리

에이전트 모델

```
class MovingEntity : public BaseGameEntity
{
protected:
  Vector2D    m_vVelocity;
  //a normalized vector pointing in the direction the entity is heading.
  Vector2D    m_vHeading;
  //a vector perpendicular to the heading vector
  Vector2D    m_vSide;
  double       m_dMass;   //VehicleMass              1.0
  //the maximum speed this entity may travel at.
  double       m_dMaxSpeed;
  //the maximum force this entity can produce to power itself
  //(think rockets and thrust)
  double        m_dMaxForce;
  //the maximum rate (radians per second)this vehicle can rotate
  double       m_dMaxTurnRate;
public:
```

```
class Vehicle : public MovingEntity
{
private:
  //a pointer to the world data. So a vehicle can access any obstacle,
  //path, wall or agent data
  GameWorld*           m_pWorld;
  //the steering behavior class
  SteeringBehavior*    m_pSteering;
public:
  Vehicle(GameWorld* world,
          Vector2D position,
          double   rotation,
          Vector2D velocity,
          double   mass,
          double   max_force,
          double   max_speed,
          double   max_turn_rate,
          double   scale);
  //updates the vehicle's position and orientation
  void        Update(double time_elapsed);
```

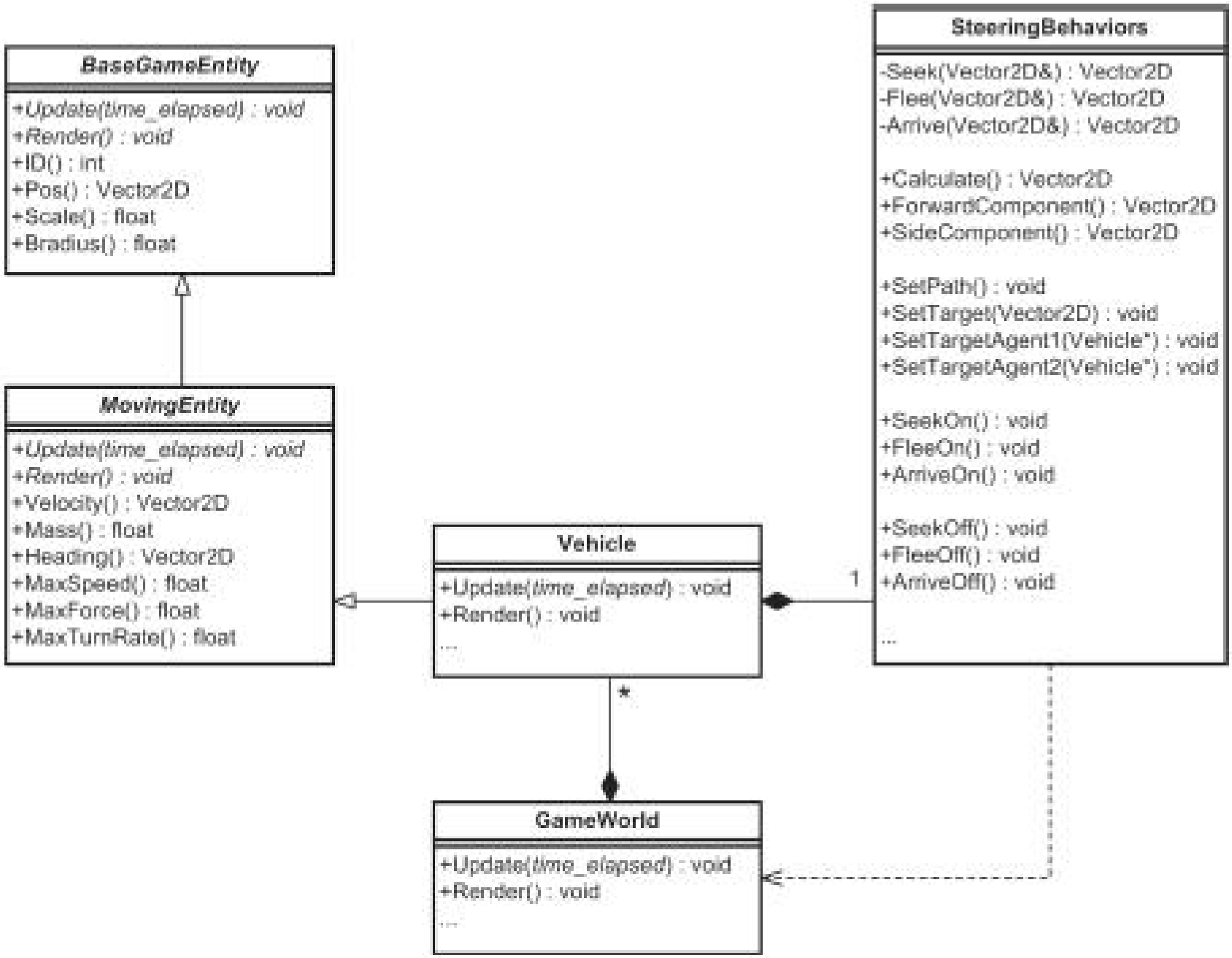


Figure 3.1. The Vehicle and SteeringBehaviors class relationships

```
void Vehicle::Update(double time_elapsed)
{
  m_dTimeElapsed = time_elapsed; //update the time elapsed
 //calculate the combined force from each steering behavior in the
  //vehicle's list
  SteeringForce = m_pSteering->Calculate();
 //Acceleration = Force/Mass
  Vector2D acceleration = SteeringForce / m_dMass;
  //update velocity
  m_vVelocity += acceleration * time_elapsed;
  //make sure vehicle does not exceed maximum velocity
  m_vVelocity.Truncate(m_dMaxSpeed);
  //update the position
  m_vPos += m_vVelocity * time_elapsed;
//update the heading if the vehicle has a non zero velocity
  if (m_vVelocity.LengthSq() > 0.00000001)  {
    m_vHeading = Vec2DNormalize(m_vVelocity);
    m_vSide = m_vHeading.Perp();
  }
 //treat the screen as a toroid
  WrapAround(m_vPos, m_pWorld->cxClient(), m_pWorld->cyClient());
```

벡터와 프레임 갱신

- main의 dt를 GameWorld, Vehicle, Calculate 순서로 전달
- 힘 / 질량으로 가속도 계산 후 속도와 위치에 dt 적용
- Heading은 단위 방향, Velocity는 방향과 속력을 함께 표현
- 이 Update에서는 MaxTurnRate로 회전량을 제한하지 않음

실습 01: 우주 화물선 출발 대결

- 임무: 같은 추진력으로 가벼운 탐사선과 무거운 화물선을 출발
- 조건: 질량 1과 4, 초기 속도 0, 같은 최대 속력과 조종력
- 실험: 힘 (100, 0), dt=0.1로 10회 갱신 후 결과 비교
- 완료: 질량 변화가 가속과 이동 거리에 주는 영향을 설명

조종 행동

- 돌진하기 (seek)

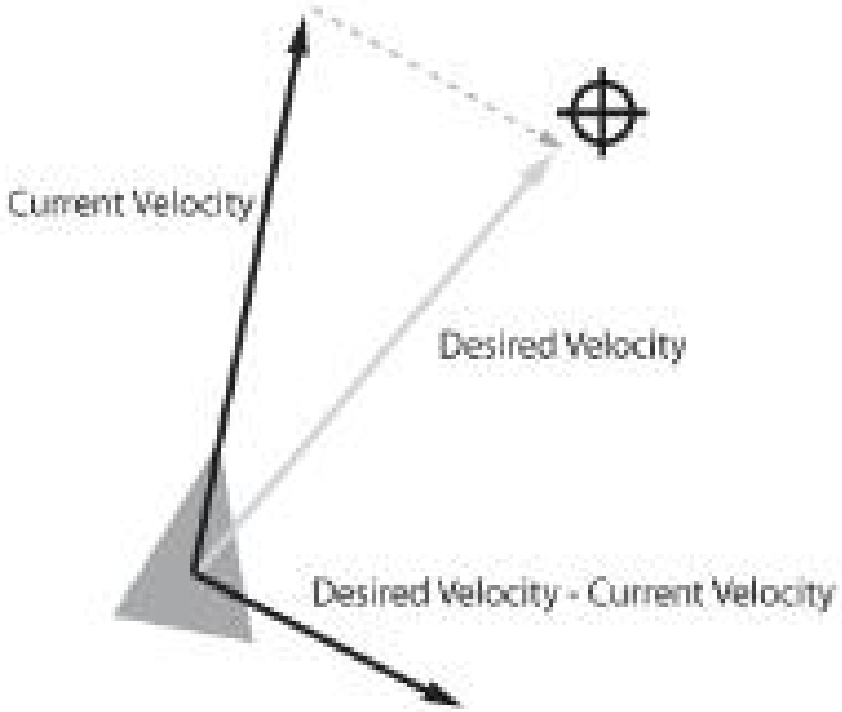


Figure 3.2. Calculating vectors for the seek behavior. The dotted vector shows how the addition of the steering force to the current velocity produces the desired result.

```
Vector2D SteeringBehavior::Seek(Vector2D TargetPos)
{
  Vector2D DesiredVelocity = Vec2DNormalize(TargetPos - m_pVehicle->Pos())
                             * m_pVehicle->MaxSpeed();

  return (DesiredVelocity - m_pVehicle->Velocity());
}
```

목적지에서 시작점을 빼서 구함

속도의 변화량(가속도) | * 질량(점질량 1로 설정됨) -> 힘을 반환

Seek의 속도 차 해석

- 목표 속도는 목표 방향의 단위 벡터에 MaxSpeed를 곱한 값
- 목표 속도에서 현재 속도를 빼 가속·제동·방향 보정을 요구
- 속도 차는 게임용 조종 신호이며 물리 단위의 힘과 구분
- 목표 근처 감속과 도착 판정은 Arrive 및 별도 정책이 담당

조종 행동

- 달아나기 (flee)

```
Vector2D SteeringBehavior::Flee(Vector2D TargetPos)
{
  //only flee if the target is within 'panic distance'. Work in distance
  //squared space.
 /* const double PanicDistanceSq = 100.0f * 100.0;
  if (Vec2DDistanceSq(m_pVehicle->Pos(), target) > PanicDistanceSq)
  {
    return Vector2D(0,0);
  }
  */

  Vector2D DesiredVelocity = Vec2DNormalize(m_pVehicle->Pos() - TargetPos)
                             * m_pVehicle->MaxSpeed();

  return (DesiredVelocity - m_pVehicle->Velocity());
}
```

현위치에서 타겟을 빼서 구함

도착하기(arrive)

```
 //Arrive makes use of these to determine how quickly a vehicle
  //should decelerate to its target
  enum Deceleration{slow = 3, normal = 2, fast = 1};

Vector2D SteeringBehavior::Arrive(Vector2D    TargetPos,
                                  Deceleration deceleration)
{
  Vector2D ToTarget = TargetPos - m_pVehicle->Pos();

  //calculate the distance to the target
  double dist = ToTarget.Length();

  if (dist > 0)
  {
    //because Deceleration is enumerated as an int, this value is required
    //to provide fine tweaking of the deceleration..
    const double DecelerationTweaker = 0.3;
```

도착하기(arrive)

```
    //calculate the speed required to reach the target given the desired
    //deceleration  dist가 가까워질수록 dist가 0이 돼서 도착 직전의 속도는 0이 됨
    double speed =  dist / ((double)deceleration * DecelerationTweaker);

    //make sure the velocity does not exceed the max
    speed = min(speed, m_pVehicle->MaxSpeed());

    //from here proceed just like Seek except we don't need to normalize
    //the ToTarget vector because we have already gone to the trouble
    //of calculating its length: dist.
    Vector2D DesiredVelocity =  ToTarget * speed / dist;

    return (DesiredVelocity - m_pVehicle->Velocity());
  }

  return Vector2D(0,0);
}
```

방향벡터

실습 02: 착륙장 정밀 주차

- 임무: 클릭한 착륙장에 도착해 멈추는 드론 구현
- 비교: Seek와 Arrive의 도착 후 진동과 정지 상태
- 조건: 목표 거리 3 미만, 속력 2 미만을 동시에 만족
- 완료: 주차 성공을 한 번만 기록하고 다음 클릭까지 대기

조종 행동

- 추격하기

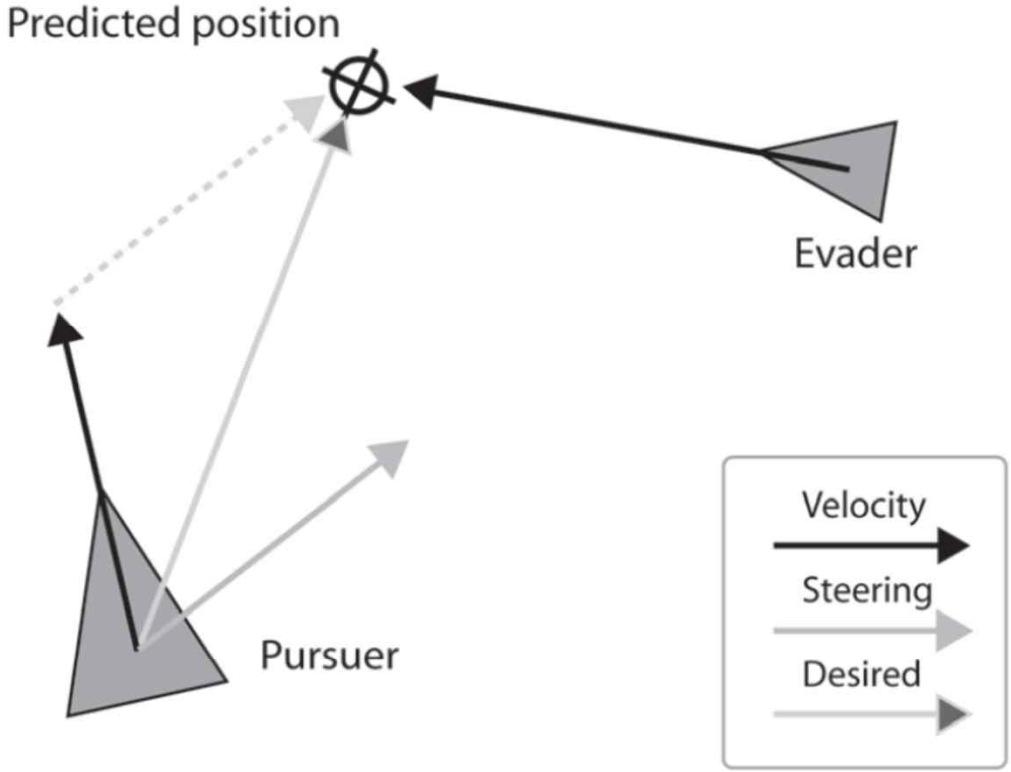


Figure 3.3. Calculating the vectors for the pursuit steering behavior. Once again, the dotted vector shows how the addition of the steering force to the current velocity produces the desired result.

```
Vector2D SteeringBehavior::Pursuit(const Vehicle* evader)
{
  //if the evader is ahead and facing the agent then we can just seek
  //for the evader's current position.
  Vector2D ToEvader = evader->Pos() - m_pVehicle->Pos();

  double RelativeHeading = m_pVehicle->Heading().Dot(evader->Heading());

  if ( (ToEvader.Dot(m_pVehicle->Heading()) > 0) &&
       (RelativeHeading < -0.95))  //acos(-0.95)=162 degs, 마주보는 상황
  {
    return Seek(evader->Pos());
  }
```

```
  //Not considered ahead so we predict where the evader will be.

   //the lookahead time is propotional to the distance between the evader
   //and the pursuer; and is inversely proportional to the sum of the
   //agent's velocities
   double LookAheadTime = ToEvader.Length() /
                          (m_pVehicle->MaxSpeed() + evader->Speed());

   //now seek to the predicted future position of the evader
   return Seek(evader->Pos() + evader->Velocity() * LookAheadTime);
}
```

도피하기(evade)

```
//------------------------------ Evade -----------------------------------
//
//  similar to pursuit except the agent Flees from the estimated future
//  position of the pursuer
//------------------------------------------------------------------------
Vector2D SteeringBehavior::Evade(const Vehicle* pursuer)
{
  /* Not necessary to include the check for facing direction this time */

  Vector2D ToPursuer = pursuer->Pos() - m_pVehicle->Pos();

  //uncomment the following two lines to have Evade only consider pursuers
  //within a 'threat range'
  const double ThreatRange = 100.0;
  if (ToPursuer.LengthSq() > ThreatRange * ThreatRange) return Vector2D();
```

```
  //the lookahead time is propotional to the distance between the pursuer
   //and the evader; and is inversely proportional to the sum of the
   //agents' velocities
   double LookAheadTime = ToPursuer.Length() /
                          (m_pVehicle->MaxSpeed() + pursuer->Speed());

   //now flee away from predicted future position of the pursuer
   return Flee(pursuer->Pos() + pursuer->Velocity() * LookAheadTime);
}
```

예측 추격과 위협 범위

- 예측 시간 = 거리 / (자신의 MaxSpeed + 상대 Speed)
- Pursuit는 미래 위치에 Seek, Evade는 그 위치에서 Flee
- 제공된 Evade는 거리 100 밖에서 조종력 0을 반환
- 등속 직선 운동의 근사이며 급회전과 장애물까지 예측하지 않음

실습

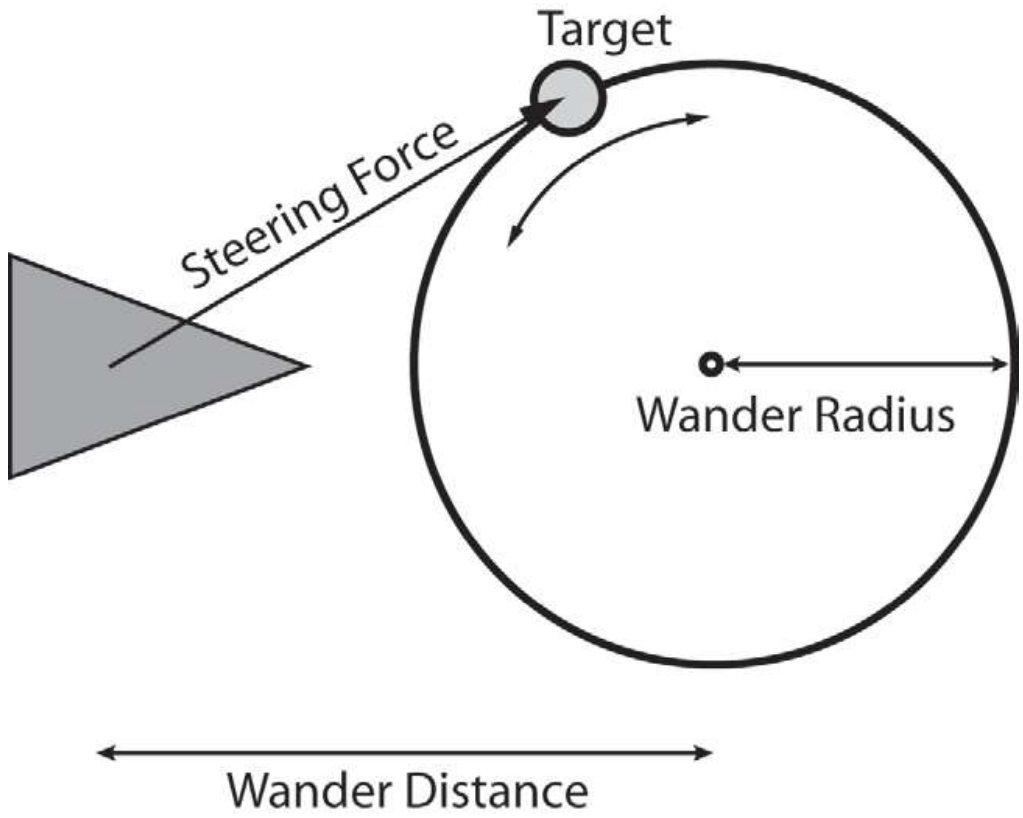
- 돌진하기, 달아나기, 도착하기, 추격하기, 도피하기
  - 각 행동 분석
  - 개별 행동 실행

실습 03: 드론 추격전

- 임무: 순찰 드론을 잡는 요격 드론과 도망가는 드론 구현
- 비교: 현재 위치 추적과 미래 위치 추적을 같은 조건에서 실험
- 도전: 도주 드론은 위협이 가까울 때만 회피
- 완료: 20초 안의 포획 여부와 최소 접근 거리를 기록

배회하기 (wander)

- 에이전트가 자신이 처한 환경에서 무작위로 걸어가는 인상을 주는 조종힘



```
Vector2D SteeringBehavior::Wander()
{
  //this behavior is dependent on the update rate, so this line must
  //be included when using time independent framerate.
  double JitterThisTimeSlice = m_dWanderJitter * m_pVehicle->TimeElapsed();

  //first, add a small random vector to the target's position
  m_vWanderTarget += Vector2D(RandomClamped() * JitterThisTimeSlice,
                              RandomClamped() * JitterThisTimeSlice);

  //reproject this new vector back on to a unit circle
  m_vWanderTarget.Normalize();
```

```
  //increase the length of the vector to the same as the radius
   //of the wander circle
   m_vWanderTarget *= m_dWanderRadius;

   //move the target into a position WanderDist in front of the agent
   Vector2D target = m_vWanderTarget + Vector2D(m_dWanderDistance, 0);

   //project the target into world space
   Vector2D Target = PointToWorldSpace(target,
                                       m_pVehicle->Heading(),
                                       m_pVehicle->Side(),
                                       m_pVehicle->Pos());

   //and steer towards it
   return Target - m_pVehicle->Pos(); // 방향 및 세기 벡터를 반환함
}
```

로컬 좌표와 월드 좌표

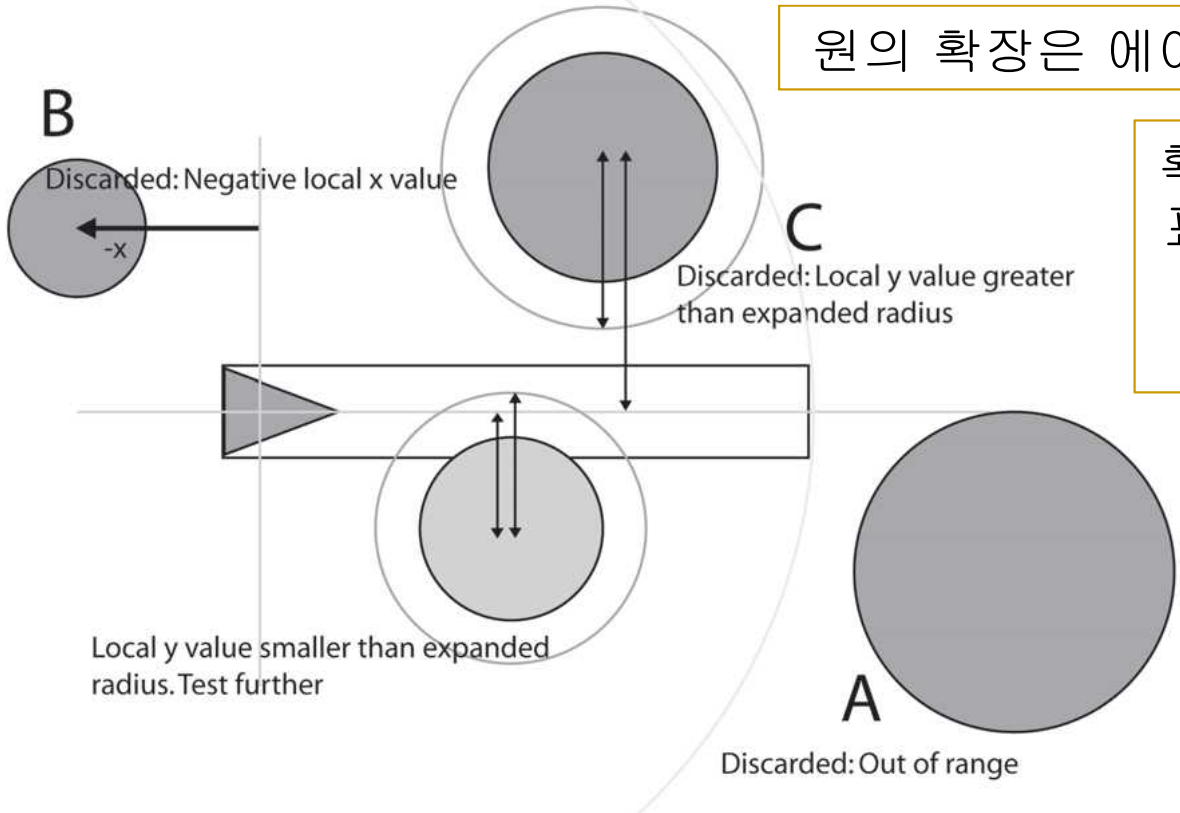
- 로컬 x축은 Heading, y축은 Side
- 점의 변환은 위치를 더하고 힘 벡터의 변환은 회전만 적용
- Wander는 이전 원 위 목표에 작은 변화를 누적
- jitter에 dt를 곱해도 프레임률별 궤적이 완전히 같지는 않음

실습 04: 침착한 경비병과 불안한 드론

- 임무: Wander 설정만으로 서로 다른 이동 성격 표현
- 비교: jitter=10과 80, 반지름과 전방 거리는 고정
- 기록: 같은 시간 동안의 이동 궤적과 방향 변화
- 완료: 어떤 변수가 흔들림과 직진 성향을 바꾸는지 설명

장애물 피하기 (obstacle avoidance)

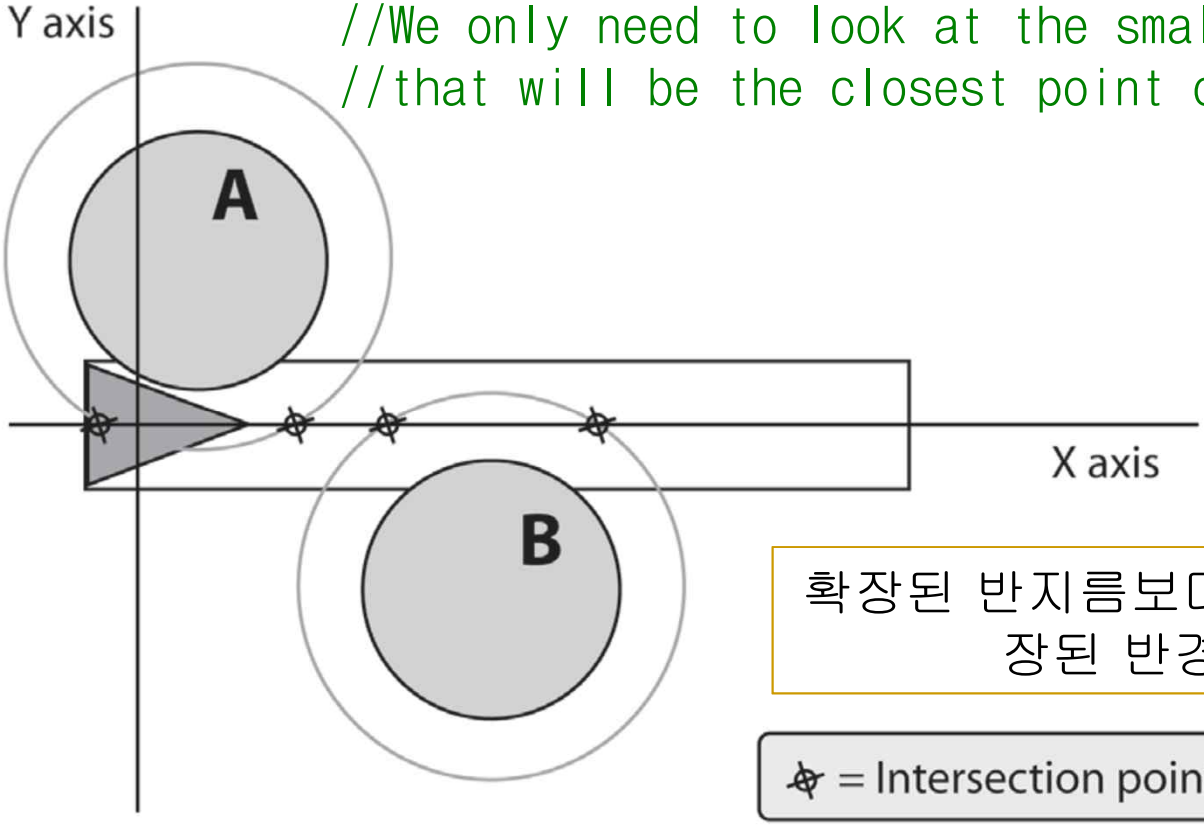
- 에이전트의 앞쪽으로 확장시켜 얻은 감지 상자인 정방형 지역 내에서 충돌이 없게
  - 감지 상자의 폭은 에이전트의 경계 반경과 동일
  - 길이는 에이전트의 현재 속도에 비례



원의 확장은 에이전트 폭의 절반만큼임

확장된 반지름보다 y좌표 절대값이 작다는 것은 확장된 반경이 x축과 겹친다는 의미

```
//now to do a line/circle intersection test. The center of the
//circle is represented by (cX, cY). The intersection points are
//given by the formula x = cX +/-sqrt(r^2-cY^2) for y=0.
//We only need to look at the smallest positive value of x because
//that will be the closest point of intersection.
```



확장된 반지름보다 y좌표 절대값이 작다는 것은 확장된 반경이 x축과 겹친다는 의미

Figure 3.8. Intersection points

```
Vector2D SteeringBehavior::ObstacleAvoidance(const
std::vector<BaseGameEntity*>& obstacles)
{
  //the detection box length is proportional to the agent's velocity
  m_dDBoxLength = Prm.MinDetectionBoxLength +
                  (m_pVehicle->Speed()/m_pVehicle->MaxSpeed()) *
                  Prm.MinDetectionBoxLength;
 //if the distance from the x axis to the object's position is less
       //than its radius + half the width of the detection box then there
       //is a potential intersection.
       double ExpandedRadius = (*curOb)->BRadius() + m_pVehicle->BRadius();

       if (fabs(LocalPos.y) < ExpandedRadius)
       {
         double cX = LocalPos.x;
         double cY = LocalPos.y;
         double SqrtPart = sqrt(ExpandedRadius*ExpandedRadius - cY*cY);
         double ip = cX - SqrtPart;
```

```
//if we have found an intersecting obstacle, calculate a steering
 //force away from it
 Vector2D SteeringForce;
 if (ClosestIntersectingObstacle)  {
   //the closer the agent is to an object, the stronger the
   //steering force should be
   double multiplier = 1.0 + (m_dDBoxLength - LocalPosOfClosestObstacle.x) /
                       m_dDBoxLength;
   //calculate the lateral force
   SteeringForce.y = (ClosestIntersectingObstacle->BRadius()-
                      LocalPosOfClosestObstacle.y)  * multiplier;
   //apply a braking force proportional to the obstacles distance from
   //the vehicle. 역방향 맞음
   const double BrakingWeight = 0.2;
   SteeringForce.x = (ClosestIntersectingObstacle->BRadius() -
                      LocalPosOfClosestObstacle.x) *
                      BrakingWeight;
 }
 //finally, convert the steering vector from local to world space
 return VectorToWorldSpace(SteeringForce,
                           m_pVehicle->Heading(), m_pVehicle->Side());
}
```

잘못됨 수정

```
//calculate the lateral force
   if (LocalPosOfClosestObstacle.y > 0) {
       SteeringForce.y = (LocalPosOfClosestObstacle.y -
              ClosestIntersectingObstacle->BRadius()) * multiplier;
   }
   else {
       SteeringForce.y = (ClosestIntersectingObstacle->BRadius() +
          LocalPosOfClosestObstacle.y) * multiplier;
   }
   //또는
   SteeringForce.y = (LocalPosOfClosestObstacle.y -
                      ClosestIntersectingObstacle->BRadius() *
   (LocalPosOfClosestObstacle.y /fabs(LocalPosOfClosestObstacle.y))) *
   multiplier;

   // 또는
   SteeringForce.y = (LocalPosOfClosestObstacle.y -
   (ClosestIntersectingObstacle->BRadius() *
       (LocalPosOfClosestObstacle.y > 0 ? 1 : -1))) * multiplier;
```

장애물 회피식 점검

- 감지 상자 반폭은 에이전트 반경, 전체 폭은 그 두 배
- 확장 반경은 장애물 반경 + 에이전트 반경
- 회피 방향은 local y의 반대쪽으로 명시하고 0인 경우도 정의
- 원본 수정 식의 y / abs(y)는 y=0에서 나눗셈 문제가 생김

벽 피하기 (wall avoidance)

- 벽과의 충돌을 피하기 위해 조종
  - 에이전트 앞에 세 개의 '촉수(feeler)'를 투사하고 검사

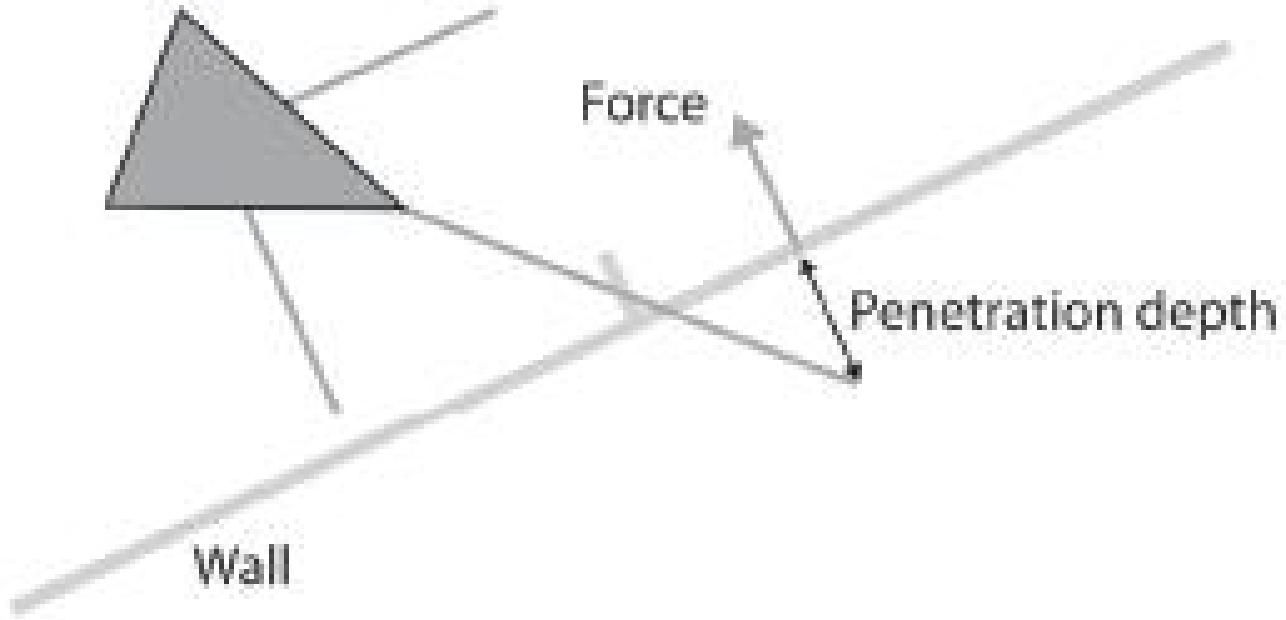


Figure 3.10. Wall avoidance

벽 회피의 촉수별 상태

- 정면 촉수와 좌우 45도 촉수로 벽 선분 교차를 검사
- 각 촉수별 최근접 벽을 원하면 반복 시작에 거리와 벽 번호 초기화
- 전역 최근접 하나를 고르면 해당 촉수 번호도 함께 저장
- 벽의 법선 방향과 촉수를 넘어선 길이로 회피력 생성

실습 05: 좌우 대칭 장애물 테스트

- 임무: 왼쪽·오른쪽·정면 장애물을 안전하게 피하는 AI 점검
- 조건: 동일한 전방 거리에서 측면 좌표만 +5, −5, 0으로 변경
- 도전: 회피 방향과 제동 방향을 수치로 확인
- 완료: 대칭 장면의 측면 힘은 반대, 정면에서도 유한한 값

끼워 넣기 (interpose)

- 두 개의 다른 에이전트 사이를 연결하는 가공의 선분의 중간 지점으로 에이전트를 이동

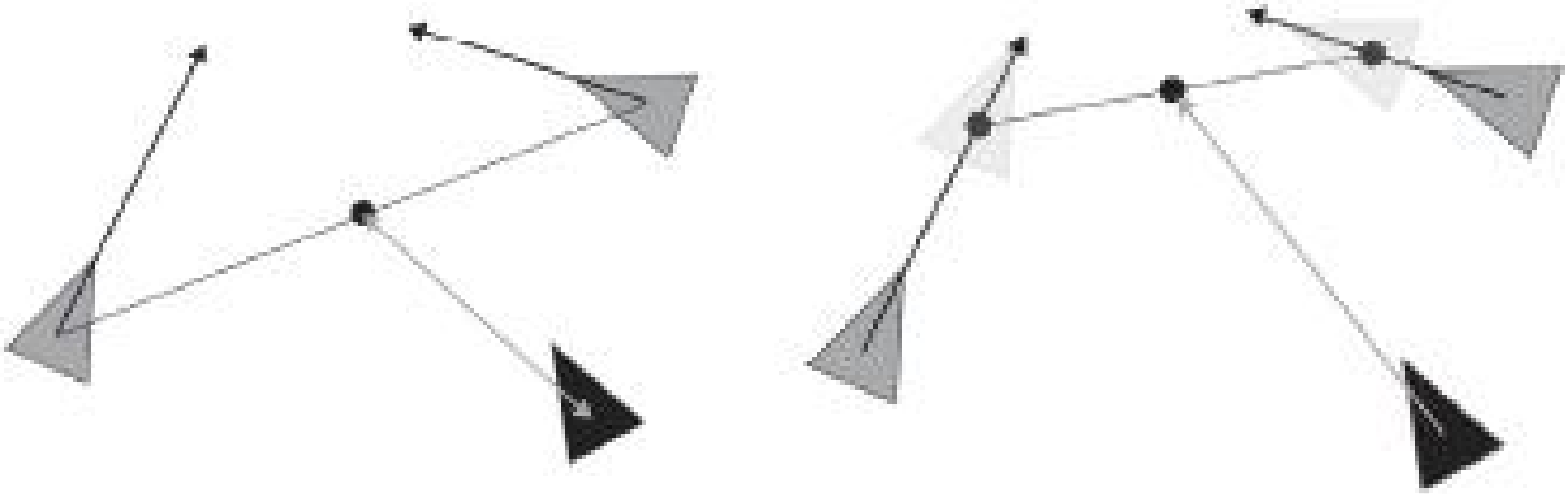


Figure 3.11. Predicting the interpose point

```
Vector2D SteeringBehavior::Interpose(const Vehicle* AgentA,
                                     const Vehicle* AgentB)
{
  //first we need to figure out where the two agents are going to be at
  //time T in the future. This is approximated by determining the time
  //taken to reach the mid way point at the current time at max speed.
  Vector2D MidPoint = (AgentA->Pos() + AgentB->Pos()) / 2.0;
  double TimeToReachMidPoint = Vec2DDistance(m_pVehicle->Pos(), MidPoint) /
                               m_pVehicle->MaxSpeed();
  //now we have T, we assume that agent A and agent B will continue on a
  //straight trajectory and extrapolate to get their future positions
  Vector2D APos = AgentA->Pos() + AgentA->Velocity() * TimeToReachMidPoint;
  Vector2D BPos = AgentB->Pos() + AgentB->Velocity() * TimeToReachMidPoint;
  //calculate the mid point of these predicted positions
  MidPoint = (APos + BPos) / 2.0;
  //then steer to Arrive at it
  return Arrive(MidPoint, fast);
}
```

숨기 (hide)

- 피하려고 하는 에이전트와 자신 사이에 항상 장애물이 있도록 하는 것

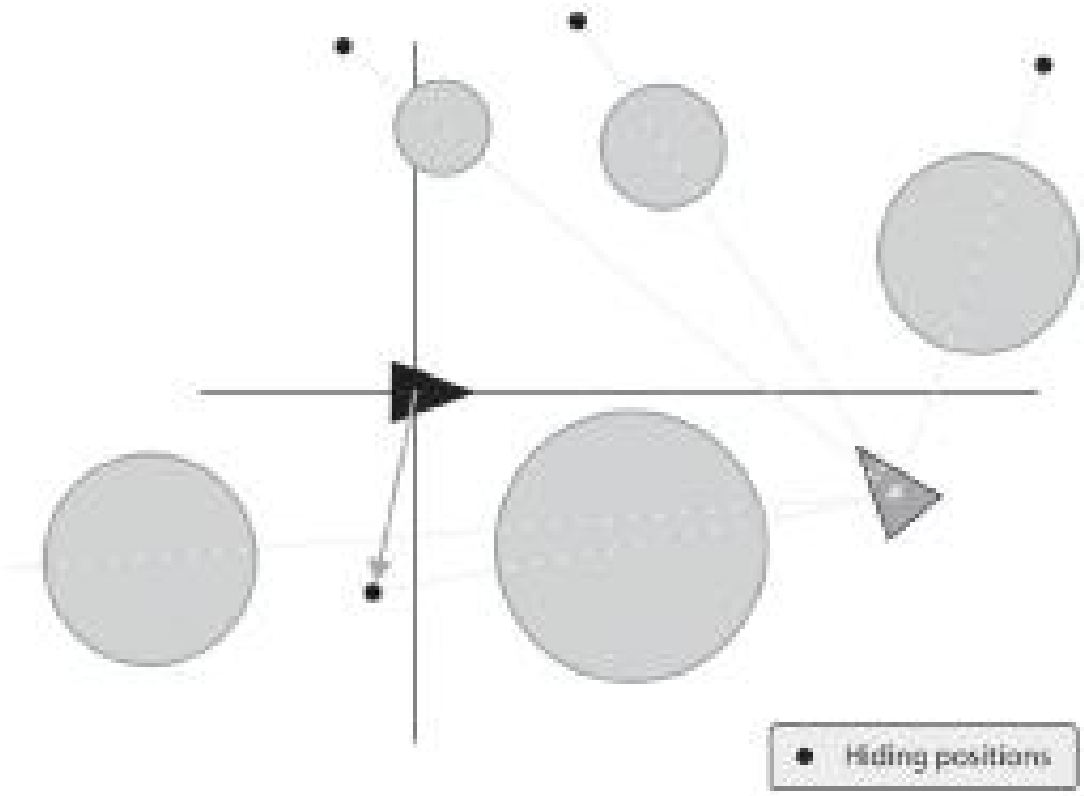


Figure 3.12. Potential hiding spots

```
Vector2D SteeringBehavior::GetHidingPosition(const Vector2D& posOb,
                                             const double     radiusOb,
                                             const Vector2D& posHunter)
{
  //calculate how far away the agent is to be from the chosen obstacle's
  //bounding radius
  const double DistanceFromBoundary = 30.0;
  double       DistAway    = radiusOb + DistanceFromBoundary;

  //calculate the heading toward the object from the hunter
  Vector2D ToOb = Vec2DNormalize(posOb - posHunter);

  //scale it to size and add to the obstacles position to get
  //the hiding spot.
  return (ToOb * DistAway) + posOb;
}
```

실습

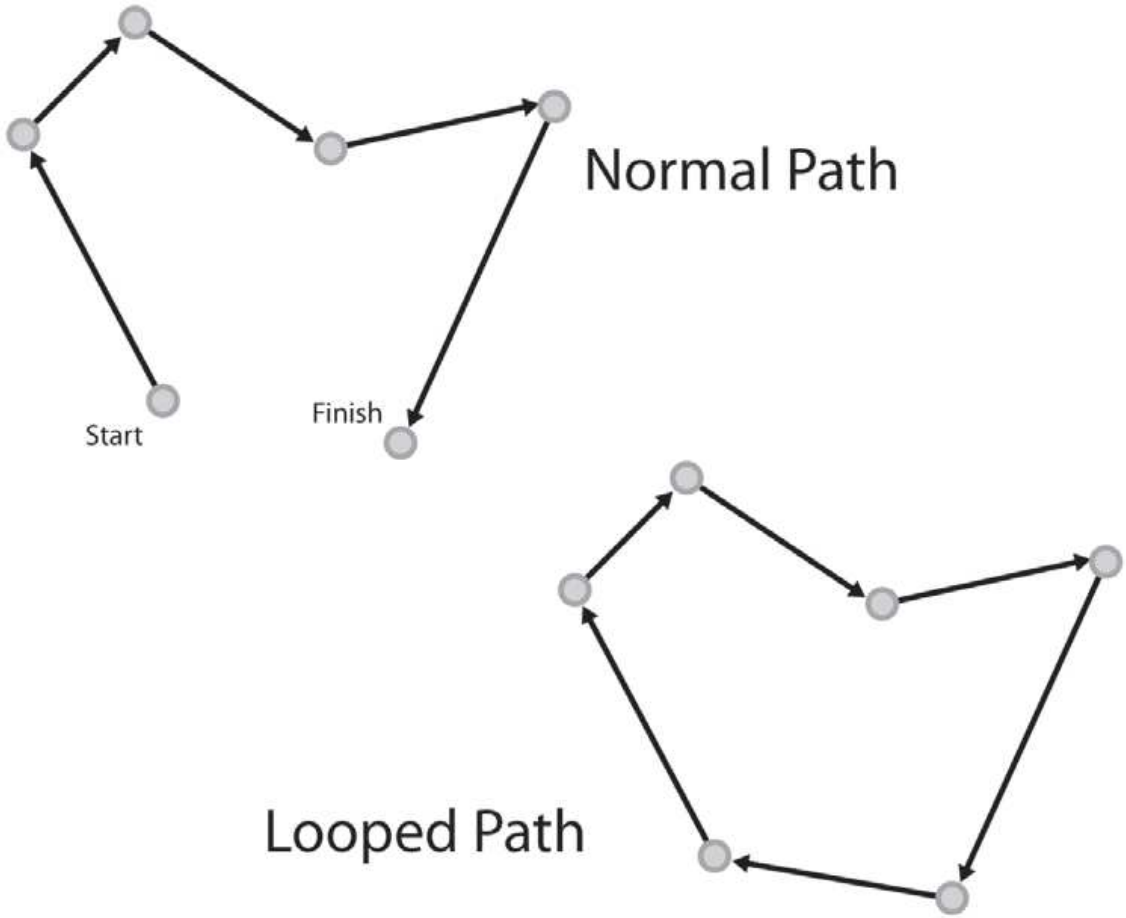
- 숨기(Hide) 수정
  - 추격자가 보이거나 지나간 n초 이내에 추격자를 봤다면 자신을 숨긴다
  - 추격자의 옆쪽이나 뒤쪽의 은신 지점들을 선호하도록 숨는다. 지금은 가장 가까운 곳에 숨음.
  - 추격자가 '위협 거리' 안에 있는지 확인하여 숨기를 실행한다

실습 06: 스텔스 잠입 미션

- 임무: 경비병을 발견하면 엄폐하고 3초 뒤 순찰에 복귀
- 조건: 마지막 목격 시각을 기억해 잠깐 시야를 잃어도 은신
- 도전: 장애물이 없을 때도 대체 행동으로 전환
- 완료: 은신 후보 선택과 복귀 시점을 로그로 확인

경로 따라가기 (path following)

- 경로를 형성하고 있는 일련의 중간지점들을 따라서 이동하도록 하는 조종힘을 제공



```
Vector2D SteeringBehavior::FollowPath()
{
  //move to next target if close enough to current target (working in
  //distance squared space)
  if(Vec2DDistanceSq(m_pPath->CurrentWaypoint(), m_pVehicle->Pos()) <
     m_dWaypointSeekDistSq)
  {
    m_pPath->SetNextWaypoint();
  }

  if (!m_pPath->Finished())
  {
    return Seek(m_pPath->CurrentWaypoint());
  }

  else
  {
    return Arrive(m_pPath->CurrentWaypoint(), normal);
  }
}
```

비순환 경로의 종료 처리

- 원본 의도는 중간 지점 Seek, 마지막 지점 Arrive
- 제공된 Finished()는 마지막 유효 지점이 아닌 iterator의 end
- 비순환 경로에서 end를 CurrentWaypoint로 읽지 않도록 개선
- 마지막 유효 지점을 유지하고 거리와 속력으로 완료 판정

실습 07: 체크포인트 배달 레이스

- 임무: 체크포인트 5개를 방문하고 마지막 배달 지점에 정지
- 규칙: 앞 지점을 방문하기 전에는 다음 지점을 완료할 수 없음
- 비교: 도달 반경 5와 20에서 코너 통과 모양 관찰
- 완료: 비순환 경로 종료 후에도 잘못된 지점 접근이 없음

오프셋 추격하기 (offset pursuit)

- 에이전트를 목표 에이전트로부터 명시된 오프셋만큼 떨어진 위치를 유지하기 위한 조종힘을 계산

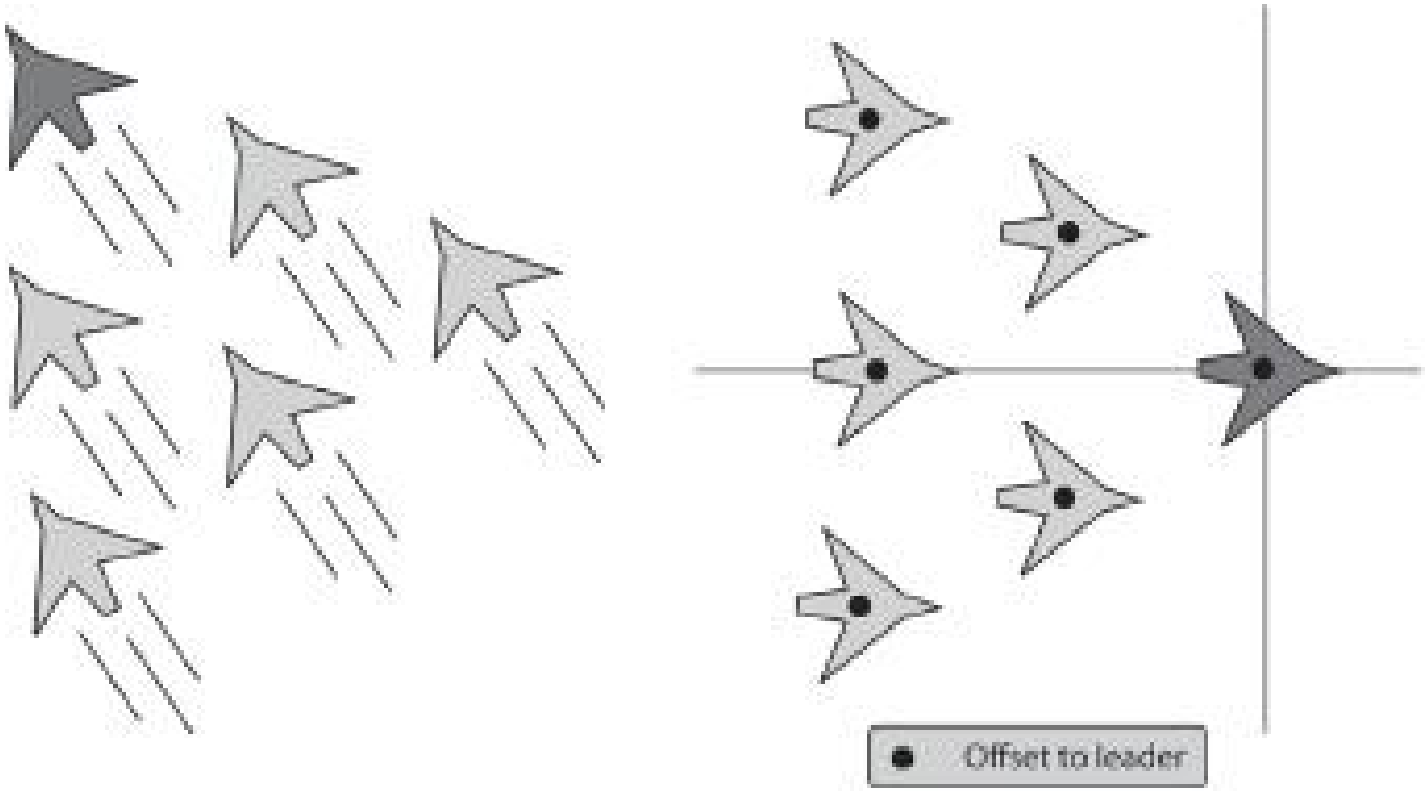


Figure 3.14. Offset pursuit. The leader is shown in dark gray.

```
Vector2D SteeringBehavior::OffsetPursuit(const Vehicle*  leader,
                                         const Vector2D offset)
{
  //calculate the offset's position in world space
  Vector2D WorldOffsetPos = PointToWorldSpace(offset,
                                              leader->Heading(),
                                              leader->Side(),
                                              leader->Pos());

  Vector2D ToOffset = WorldOffsetPos - m_pVehicle->Pos();

  //the lookahead time is propotional to the distance between the leader
  //and the pursuer; and is inversely proportional to the sum of both
  //agent's velocities
  double LookAheadTime = ToOffset.Length() /
                         (m_pVehicle->MaxSpeed() + leader->Speed());

  //now Arrive at the predicted future position of the offset
  return Arrive(WorldOffsetPos + leader->Velocity() * LookAheadTime, fast);
}
```

실습

- 10개의 개체가 편대를 갖춰서 움직이게 하기

실습 08: 드론 에어쇼

- 임무: 리더 1개와 추종자 9개로 V자 편대 비행
- 도전: 10초 뒤 일렬 편대로 바꾸고 다시 대형 유지
- 관찰: 리더가 회전할 때 편대도 함께 회전하는가?
- 완료: 개체별 로컬 offset과 편대 복구 시간을 제시

집단 행동 (group behavior)

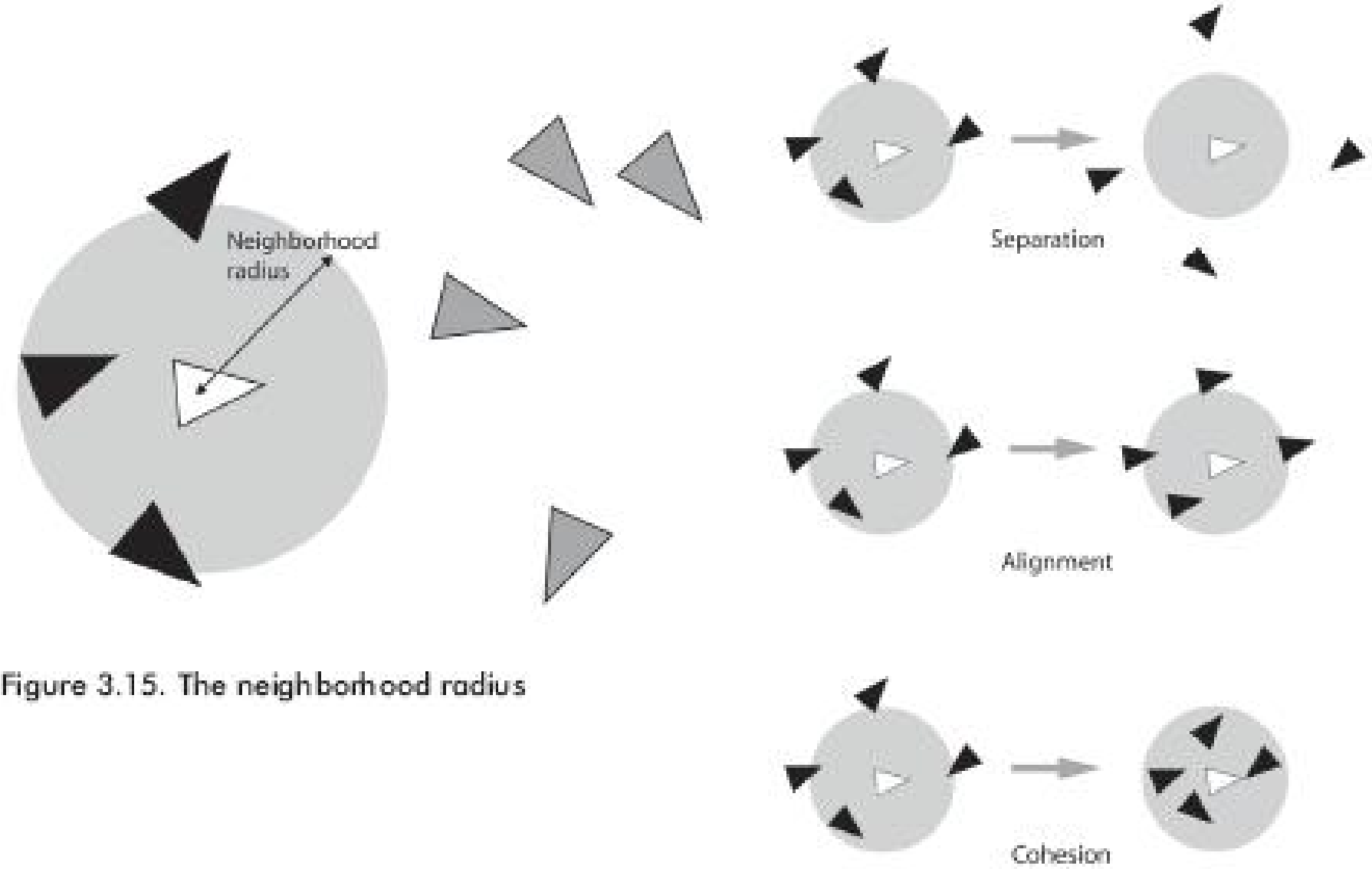


Figure 3.15. The neighborhood radius

```
Vector2D SteeringBehavior::Separation(const vector<Vehicle*> &neighbors)
{
  Vector2D SteeringForce;
  for (unsigned int a=0; a<neighbors.size(); ++a)  {
    //make sure this agent isn't included in the calculations and that
    //the agent being examined is close enough. ***also make sure it doesn't
    //include the evade target ***
    if((neighbors[a] != m_pVehicle) && neighbors[a]->IsTagged() &&
      (neighbors[a] != m_pTargetAgent1))    {
      Vector2D ToAgent = m_pVehicle->Pos() - neighbors[a]->Pos();

      //scale the force inversely proportional to the agents distance
      //from its neighbor.
      SteeringForce += Vec2DNormalize(ToAgent)/ToAgent.Length();
    }
  }
  return SteeringForce;
}
// 분리를 호출하기 전에, 에이전트의 이웃 지역 내에 놓여 있는 모든 에이전트들
을 Tag해 놓음
```

```
Vector2D SteeringBehavior::Alignment(const vector<Vehicle*>& neighbors)
{
  //used to record the average heading of the neighbors
  Vector2D AverageHeading;
  //used to count the number of vehicles in the neighborhood
  int    NeighborCount = 0;
  //iterate through all the tagged vehicles and sum their heading vectors
  for (unsigned int a=0; a<neighbors.size(); ++a)  {
    //make sure *this* agent isn't included in the calculations and that
    //the agent being examined  is close enough ***also make sure it doesn't
    //include any evade target ***
    if((neighbors[a] != m_pVehicle) && neighbors[a]->IsTagged() &&
      (neighbors[a] != m_pTargetAgent1))    {
      AverageHeading += neighbors[a]->Heading();
      ++NeighborCount;    }  }
  //if the neighborhood contained one or more vehicles, average their
  //heading vectors.
  if (NeighborCount > 0)  {
    AverageHeading /= (double)NeighborCount;
    AverageHeading -= m_pVehicle->Heading();  }
  return AverageHeading;
}
```

```
//---------------------------- Cohesion ------------------------------
//
//  returns a steering force that attempts to move the agent towards the
//  center of mass of the agents in its immediate area
//--------------------------------------------------------------------
Vector2D SteeringBehavior::Cohesion(const vector<Vehicle*> &neighbors)
{
  //first find the center of mass of all the agents
  Vector2D CenterOfMass, SteeringForce;
  int NeighborCount = 0;
  //iterate through the neighbors and sum up all the position vectors
  for (unsigned int a=0; a<neighbors.size(); ++a)  {
    //make sure *this* agent isn't included in the calculations and that
    //the agent being examined is close enough ***also make sure it doesn't
    //include the evade target ***
    if((neighbors[a] != m_pVehicle) && neighbors[a]->IsTagged() &&
      (neighbors[a] != m_pTargetAgent1))    {
      CenterOfMass += neighbors[a]->Pos();
      ++NeighborCount;
    }
  }
```

```
  if (NeighborCount > 0)
   {
     //the center of mass is the average of the sum of positions
     CenterOfMass /= (double)NeighborCount;

     //now seek towards that position
     SteeringForce = Seek(CenterOfMass);
   }

  //the magnitude of cohesion is usually much larger than separation or
  //allignment so it usually helps to normalize it.
  return Vec2DNormalize(SteeringForce);
}
```

실습

- 배회하기, 장애물 피하기, 벽 피하기, 끼워넣기, 숨기기, 경로 따라가기, 오프셋 추격하기, 무리짓기
  - 각 행동 분석 후
- 무리짓기 변경
  - 무리짓기에 새로운 행동을 추가

실습 09: 군중 대피 시뮬레이션

- 임무: 흩어진 50개 개체가 클릭한 집결 지점으로 이동
- 실험: Separation과 Cohesion 가중치를 하나씩 변경
- 기록: 제한 시간 안의 도착 수와 개체 간 최소 간격
- 완료: 도착 속도와 과밀 방지 사이의 조정 근거 제시

조종 행동들을 조합하기

- 가중 절단된 합
  - 단순, 각 조종 행동에 가중치를 곱하고 모두 더함
  - 상충하는 힘들의 조정이 어려움
- 우선순위와 함께 가중 절단된 연속합
  - 우선순위가 주어진 가중치를 사용
- 우선순위가 주어진 디더링
  - 다른 방법보다 CPU 시간을 적게 요구
  - 정확도가 떨어짐

조합 순서와 실제 가중치

- 가중 합은 최종 합을 절단하고 우선순위 방식은 순서대로 누적
- 가중치는 힘의 크기, 우선순위는 처리 순서와 예산 접근 순서
- 설정의 힘과 가중치는 SteeringForceTweaker를 곱해 사용
- 원본 prioritized의 OffsetPursuit에는 가중치 곱셈이 빠져 있음

```
Vector2D SteeringBehavior::Calculate()
{
  //reset the steering force
  m_vSteeringForce.Zero();
 switch (m_SummingMethod)  {
  case weighted_average:
    m_vSteeringForce = CalculateWeightedSum(); break;
  case prioritized:
    m_vSteeringForce = CalculatePrioritized(); break;
  case dithered:
    m_vSteeringForce = CalculateDithered();break;
  default:m_vSteeringForce = Vector2D(0,0);
  }//end switch
  return m_vSteeringForce;
}
```

```
//---------------------- CalculateWeightedSum -------------------------
//
//  this simply sums up all the active behaviors X their weights and
//  truncates the result to the max available steering force before
//  returning
//---------------------------------------------------------------------
Vector2D SteeringBehavior::CalculateWeightedSum()
{
  if (On(wall_avoidance))
  {
    m_vSteeringForce += WallAvoidance(m_pVehicle->World()->Walls()) *
                        m_dWeightWallAvoidance;
  }
  //.....
  m_vSteeringForce.Truncate(m_pVehicle->MaxForce());
  return m_vSteeringForce;
}
```

```
//---------------------- CalculatePrioritized -----------------------------
//
//  this method calls each active steering behavior in order of priority
//  and acumulates their forces until the max steering force magnitude
//  is reached, at which time the function returns the steering force
//  accumulated to that  point
//-------------------------------------------------------------------------
Vector2D SteeringBehavior::CalculatePrioritized()
{
  Vector2D force;
   if (On(wall_avoidance))
  {
    force = WallAvoidance(m_pVehicle->World()->Walls()) *
            m_dWeightWallAvoidance;
    if (!AccumulateForce(m_vSteeringForce, force)) return m_vSteeringForce;
  } // ...
 return m_vSteeringForce;
}
```

```
//---------------------- AccumulateForce ----------------------------------
//
//  This function calculates how much of its max steering force the
//  vehicle has left to apply and then applies that amount of the
//  force to add.
//-------------------------------------------------------------------------
bool SteeringBehavior::AccumulateForce(Vector2D &RunningTot,
                                       Vector2D ForceToAdd)
{
  //calculate how much steering force the vehicle has used so far
  double MagnitudeSoFar = RunningTot.Length();
  //calculate how much steering force remains to be used by this vehicle
  double MagnitudeRemaining = m_pVehicle->MaxForce() - MagnitudeSoFar;
  //return false if there is no more force left to use
  if (MagnitudeRemaining <= 0.0) return false;
  //calculate the magnitude of the force we want to add
  double MagnitudeToAdd = ForceToAdd.Length();
```

```
  //if the magnitude of the sum of ForceToAdd and the running total
   //does not exceed the maximum force available to this vehicle, just
   //add together. Otherwise add as much of the ForceToAdd vector is
   //possible without going over the max.
   if (MagnitudeToAdd < MagnitudeRemaining)
   {
     RunningTot += ForceToAdd;
   }
   else
   {
     //add it to the steering force
     RunningTot += (Vec2DNormalize(ForceToAdd) * MagnitudeRemaining);
   }
   return true;
}
```

실습 10: 보물과 위험 사이의 줄다리기

- 임무: 보물 접근과 위험 회피가 충돌할 때의 결정을 예측
- 조건: 회피력 (−12, 0), 접근력 (8, 0), MaxForce=10
- 비교: 가중 합, 회피 우선, 접근 우선의 최종 힘 계산
- 완료: 힘의 크기와 처리 순서가 만드는 차이를 설명

```
//---------------------- CalculateDithered -----------------------------
//  this method sums up the active behaviors by assigning a probabilty
//  of being calculated to each behavior. It then tests the first priority
//  to see if it should be calculated this simulation-step. If so, it
//  calculates the steering force resulting from this behavior. If it is
//  more than zero it returns the force. If zero, or if the behavior is
//  skipped it continues onto the next priority, and so on.
//----------------------------------------------------------------------
Vector2D SteeringBehavior::CalculateDithered()
{
  //reset the steering force
   m_vSteeringForce.Zero();
  if (On(wall_avoidance) && RandFloat() < Prm.prWallAvoidance)  {
    m_vSteeringForce = WallAvoidance(m_pVehicle->World()->Walls()) *
                       m_dWeightWallAvoidance / Prm.prWallAvoidance;
    if (!m_vSteeringForce.isZero())    {
      m_vSteeringForce.Truncate(m_pVehicle->MaxForce());
      return m_vSteeringForce;
    }
  }
 return m_vSteeringForce;
}
```

Dithered와 설정 파싱

- 확률 p로 평가하고 선택 시 1/p로 힘을 보정한 뒤 절단
- 일부 행동은 이 함수에 구현되지 않아 모드별 결과가 다를 수 있음
- params.ini의 float prWallAvoidance 0.5는 로더가 잘못 읽는 형식
- float를 제거하고 실행 중 읽은 확률이 0.5인지 확인

실습 11: 절전형 경비 AI

- 임무: 생각하는 횟수를 줄여 계산량과 회피 반응 비교
- 실험: 벽 회피 확률 p=0.2와 0.8로 각각 1,000회 평가
- 기록: 실제 함수 호출 수와 선택된 힘의 크기
- 완료: 확률 보정의 효과와 안전상의 한계를 설명

대량의 에이전트에 대처: 공간 분할하기

- 셀 공간 분할하기 (cell-space partitioning)
- 상자 공간 분할하기 (bin-space partitioning)
  - 2차원 공간은 많은 수의 셀(bin)들로 쪼개진다
  - 각 셀은 자신이 포함하고 있는 모든 엔티티들을 가리키고 있는 포인터들의 리스트를 가짐
  - 엔티티가 위치를 변화시킬 때마다 갱신
- 어떤 셀들이 에이전트의 인근 범위 내에 놓여 있는지를 결정
  - 이런 셀들에 포함되어 있는 에이전트를 상대로 검사

대량의 에이전트에 대처: 공간 분할하기

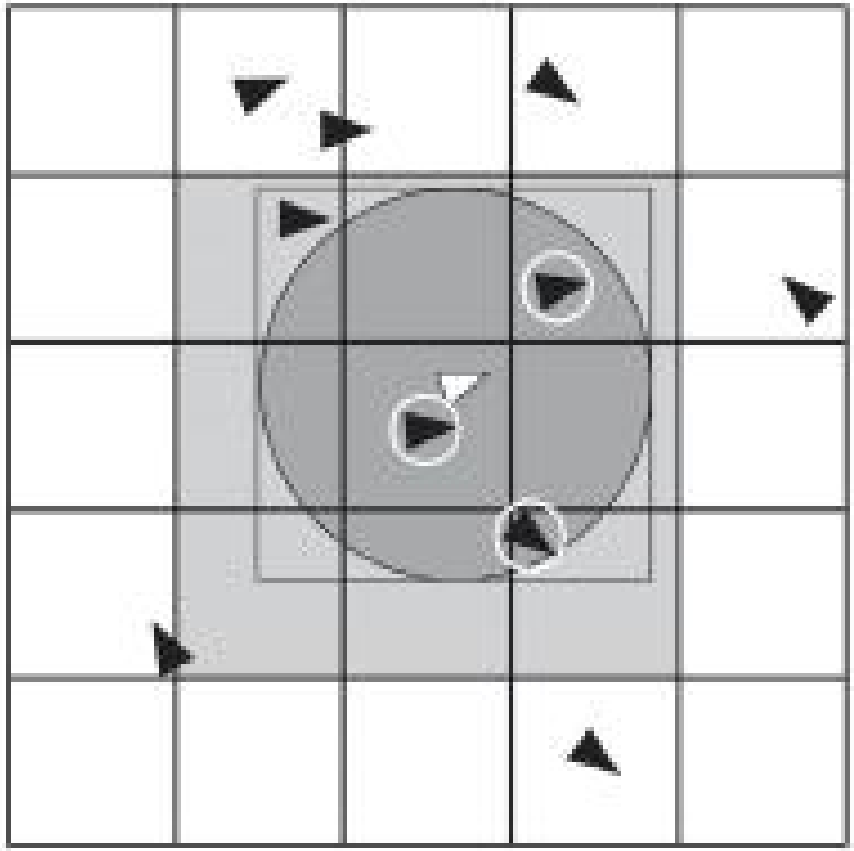


Figure 3.18. Cell-space partitioning. The circled vehicles are those within the white vehicle's neighborhood region.

```
//-----------------------------------------------------------------------
//  defines a cell containing a list of pointers to entities
//-----------------------------------------------------------------------
template <class entity>
struct Cell
{
  //all the entities inhabiting this cell
  std::list<entity>    Members;
  //the cell's bounding box (it's inverted because the Window's default
  //co-ordinate system has a y axis that increases as it descends)
  InvertedAABBox2D     BBox;
  Cell(Vector2D topleft,
       Vector2D botright):BBox(InvertedAABBox2D(topleft, botright))
  {}
};
```

```
template <class entity>
class CellSpacePartition
{
  //the required amount of cells in the space
  std::vector<Cell<entity> >                m_Cells;
  //this is used to store any valid neighbors when an agent searches
  //its neighboring space
  std::vector<entity>                       m_Neighbors;
  //this iterator will be used by the methods next and begin to traverse
  //through the above vector of neighbors
  typename std::vector<entity>::iterator    m_curNeighbor;
  //the width and height of the world space the entities inhabit
  double  m_dSpaceWidth;
  double  m_dSpaceHeight;
  //the number of cells the space is going to be divided up into
  int     m_iNumCellsX;
  int     m_iNumCellsY;
  double  m_dCellSizeX;
  double  m_dCellSizeY;
  //given a position in the game space this method determines the
  //relevant cell's index
  inline int  PositionToIndex(const Vector2D& pos)const;
```

```
//------------------------ CalculateNeighbors -----------------------------
//  This must be called to create the vector of neighbors.This method
//  examines each cell within range of the target, If the
//  cells contain entities then they are tested to see if they are situated
//  within the target's neighborhood region. If they are they are added to
//  neighbor list
//-------------------------------------------------------------------------
template<class entity>
void CellSpacePartition<entity>::CalculateNeighbors(Vector2D TargetPos,
                                                    double   QueryRadius)
{
  //create an iterator and set it to the beginning of the neighbor vector
  std::vector<entity>::iterator curNbor = m_Neighbors.begin();
  //create the query box that is the bounding box of the target's query
  //area
  InvertedAABBox2D QueryBox(TargetPos - Vector2D(QueryRadius, QueryRadius),
                            TargetPos + Vector2D(QueryRadius, QueryRadius));
```

```
//iterate through each cell and test to see if its bounding box overlaps
 //with the query box. If it does and it also contains entities then
 //make further proximity tests.
 std::vector<Cell<entity> >::iterator curCell;
 for (curCell=m_Cells.begin(); curCell!=m_Cells.end(); ++curCell)  {
   //test to see if this cell contains members and if it overlaps the
   //query box
   if (curCell->BBox.isOverlappedWith(QueryBox) &&
      !curCell->Members.empty())    {
     //add any entities found within query radius to the neighbor list
     std::list<entity>::iterator it = curCell->Members.begin();
     for (it; it!=curCell->Members.end(); ++it)      {
       if (Vec2DDistanceSq((*it)->Pos(), TargetPos) <
           QueryRadius*QueryRadius)        {
         *curNbor++ = *it;
       }
     }
   }
 }//next cell
 //mark the end of the list with a zero.
 *curNbor = 0;
}
```

이웃 버퍼와 검색 조건

- 원본 셀 질의는 자신도 결과에 포함하고 Plus에서 제외
- 모든 개체가 이웃이면 종결 표식용 공간이 하나 더 필요
- 일반 검색과 셀 검색의 반경·대상 제외 조건이 서로 다름
- 동일한 이웃을 고르는지 확인한 뒤 성능 비교

실습 12: 초밀집 스폰 버그 사냥

- 임무: 한 지점에 몰려 생성된 20개 개체를 안정적으로 처리
- 조건: 감지 반경 안에 모든 개체가 포함되는 상황 구성
- 점검: Separation의 거리 0과 이웃 버퍼의 마지막 기록
- 완료: NaN과 범위 초과 없이 모든 이웃을 처리

```
Vector2D SteeringBehavior::Calculate()
{
  //reset the steering force
  m_vSteeringForce.Zero();
  //use space partitioning to calculate the neighbours of this vehicle
  //if switched on. If not, use the standard tagging system
  if (!isSpacePartitioningOn())  {
    //tag neighbors if any of the following 3 group behaviors are switched on
    if (On(separation) || On(allignment) || On(cohesion))    {
      m_pVehicle->World()->TagVehiclesWithinViewRange(m_pVehicle,
m_dViewDistance);
    }
  }
  else  {
    //calculate neighbours in cell-space if any of the following 3 group
    //behaviors are switched on
    if (On(separation) || On(allignment) || On(cohesion))    {
      m_pVehicle->World()->CellSpace()->CalculateNeighbors(m_pVehicle->Pos(),
m_dViewDistance);
    }
  }
```

```
class GameWorld
{
 //a container of all the moving entities
  std::vector<Vehicle*>          m_Vehicles;
  //any obstacles
  std::vector<BaseGameEntity*>  m_Obstacles;
 CellSpacePartition<Vehicle*>* m_pCellSpace;
 void  TagVehiclesWithinViewRange(BaseGameEntity* pVehicle, double range)
  {
    TagNeighbors(pVehicle, m_Vehicles, range);
  }
 void  TagObstaclesWithinViewRange(BaseGameEntity* pVehicle, double range)
  {
    TagNeighbors(pVehicle, m_Obstacles, range);
  }
}
```

```
//----------------------- TagNeighbors --------------------------------
//  tags any entities contained in a std container that are within the
//  radius of the single entity parameter
//---------------------------------------------------------------------
template <class T, class conT>
void TagNeighbors(const T& entity, conT& ContainerOfEntities, double radius)
{
  //iterate through all entities checking for range
  for (typename conT::iterator curEntity = ContainerOfEntities.begin();
       curEntity != ContainerOfEntities.end(); ++curEntity)  {
    //first clear any current tag
    (*curEntity)->UnTag();
    Vector2D to = (*curEntity)->Pos() - entity->Pos();
    //the bounding radius of the other is taken into account by adding it
    //to the range
    double range = radius + (*curEntity)->BRadius();
    //if entity within range, tag for further consideration. (working in
    //distance-squared space to avoid sqrts)
    if ( ((*curEntity) != entity) && (to.LengthSq() < range*range))    {
      (*curEntity)->Tag();
    }
  }//next entity
}
```

실습

- 공간 분할(cell partitioning) 여부에 따른 무리짓기(flocking) 작동 방법 파악

```
//these next three can be combined for flocking behavior (wander is
//also a good behavior to add into this mix)
 if (!isSpacePartitioningOn())  {
   if (On(separation))    {
     force = Separation(m_pVehicle->World()->Agents()) * m_dWeightSeparation;
     if (!AccumulateForce(m_vSteeringForce, force)) return m_vSteeringForce;
   }
else
 {
   if (On(separation))    {
     force = SeparationPlus(m_pVehicle->World()->Agents()) *
m_dWeightSeparation;
     if (!AccumulateForce(m_vSteeringForce, force)) return m_vSteeringForce;
   }
 }
```

공간 분할의 성능 해석

- 전체 순회는 개체마다 N개 후보를 검사해 대략 N^2 비용
- 현 셀 구현은 질의마다 모든 C개 셀의 겹침을 확인
- 분산·밀집 장면, 감지 반경, 셀 수를 구분해 측정
- FPS 한순간 값보다 프레임 시간과 후보 수로 비교

실습 13: 1,000명 군중의 성능 실험

- 임무: 300명과 1,000명의 움직임을 유지하며 검색 비용 비교
- 비교: 공간 분할 OFF와 ON, 같은 감지 반경과 행동
- 기록: 이웃 후보 수와 프레임 시간의 중앙값
- 완료: 분산·밀집 배치에서 결과가 달라지는 이유 설명

실습

- 엄마 거위 뒤를 따라가는 새끼 거위들을 구현
  - 엄마 거위를 조정
  - 새끼들은 흩어져 놀고 있다가 엄마 거위가 지나가면 한 마리씩 일렬로 따라감

실습 14: 구조 로봇 기차

- 임무: 흩어져 배회하던 로봇이 지나가는 리더에게 차례로 합류
- 규칙: 새 로봇은 리더가 아니라 행렬의 마지막 로봇을 추종
- 도전: 자기 자신 추종과 중복 합류를 방지
- 완료: 합류 순서와 각 로봇의 선행 개체를 출력

마우스 커서 활성화

- void GameWorld::Render()
  - //#define CROSSHAIR -> #define CROSSHAIR